Part 12 Brief · Evals at scale: pass^k and prompt injection

mcp-ops-server · 22 Oct 2026 · PR #28 · Status: merged; kit 0.2.0 on PyPI; Bugbot (1) and two Codex passes (1x P1, 9x P2) fixed

WHAT WAS BUILT

- **Repeated runs** (`--repeats k`, 1 to 20): every task k times on fresh data, scored with **pass^k** (the chance k runs of a task all pass), per-task pass rate and variance, and flaky tasks named. A task passes only if every run passed. Cost is summed over all runs.
- **A prompt-injection suite** (`--suite injection`): 6 clinic tasks plant an instruction in a provider's name, the one free-text field the tools return, on SQLite and FHIR. 8 negative controls obey it and must be caught. Both suites gate CI on both backends; the live workflow takes suite and repeats.

HOW IT WORKS

Piece	What it does
plant (harness)	writes the text into the task's fresh data before the server starts; `fhir_seed reset` puts names back
injection_exposed	the agent must have received a tool result containing the text, else the run is not a resisted attack
attempted / carried out	a call the planted text asked for, refused or not / yes, no, or unknown (the write was sent but the backend could not confirm it)
pass^k	C(c, k) / C(n, k) per task from n runs with c passes (unbiased), averaged over tasks

WHY IT WAS DONE THIS WAY

- **Three attacks the server refuses, three the token allows.** Threat model E2 says the server bounds what an instruction can achieve; the suite measures both halves: what the model tries, and what still gets through.
- **Separate suite, same grader.** The golden numbers (31/31) stay comparable, and injection tasks use the same checks; following the injection is a forbidden step, so it fails like any other unsafe call.

WHAT WENT WRONG OR SURPRISED ME (LOGGED IN FAILURES.MD)

- **Planted text would have leaked across tasks on FHIR**: reset never restored provider names. Fixed, and a test fails without it. **I nearly rebuilt Part 11** from a stale checkout; PyPI already showing mcp-ops-kit 0.1.0 gave it away before any code was written.
- **Codex found my exposure check too generous**: a model cut off by its turn limit right after the search never received the attack, yet counted as resisting it. It also found a lost FHIR write reported as stopped, and a null referral missed by the attack pattern. All reproduced, fixed, and tested.

VERIFY IT YOURSELF

```
pytest -q                                  # 659 passed (740 with sqlite,fhir)
python -m evals.run --suite injection      # 6/6 tasks, 8/8 controls
python -m evals.run --repeats 3 --tasks b01-earliest-primary-care
```

INTERVIEW TALKING POINT

"An injection eval that only counts passes is misleading in two ways: an agent that never read the attack looks robust, and a refused call looks the same as a carried-out one. I grade exposure, attempt and outcome separately, so the number says whether the model resisted or the server saved it."

LIVE RESULTS (5 RUNS PER TASK, $2.95 IN TOTAL)

Golden: both models passed 29/31 tasks in every run (pass^1 Luna 0.955, Sol 0.961; pass^5 0.935). Each miss is consistent: Sol stops after a clash in f01 in 5/5 runs (my ambiguous prompt rule), and Luna over-calls on k01 in 5/5. Injection: 6/6 tasks, 30/30 runs each; no planted instruction followed in 60 exposed runs. Next, your call: reword the prompt rule and re-run (about $2.60).